

Low-Level Design (LLD) - Soil Temperature Monitor

(Python Core & Pandas DataFrames)

Difficulty Level: Medium | Total Marks: 20

Design Format: 1 Class | 7 Methods | 7 Test Cases

****(Marks allocation details in "Test Cases & Marks Allocation" section)****

Summary of Design Requirements

Implement a class 'SoilTempMonitor' to track and analyze soil temperature readings using Pandas DataFrames. The class focuses on data cleaning, identify peak health readings, and calculating medical trends.

The class should:

- Load soil temperature logs from a CSV file.
- Handle missing temperature values by cleaning the dataset.
- Provide statistical insights such as averages and extreme values.
- Monitor risk levels by identifying and counting high-temperature sensors.

Concepts Tested

- DataFrame state management and schemas
- Data cleaning (NaN handling)
- Aggregation and Grouping (max, groupby, mean)
- Boolean indexing and filtering
- Unique value extraction and counting

Problem Statement

Design a health monitoring tool 'SoilTempMonitor' to analyze soil temperature data. The system starts by loading 'temperature.csv' containing sensor vitals. The monitor must be capable of cleaning incomplete records, identifying peak readings, and generating reports on sensor averages and risk categories.

Note: The foundational methods for initialization and data loading (Method 1 & 2) are pre-provided.

Operations (Methods)

1. Initialize Monitor [0 Marks]

def __init__(self):

- Logic: Initialize 'self.df = None'.

2. Read Heart Data [0 Marks]

def read_data(self, file_path: str):

- Logic: Load the CSV file into the Pandas DataFrame 'self.df'.

3. Clean Records [4 Marks]

def clean_records(self) -> int:

- Logic: Drop rows where 'Temperature' is NaN. Update the internal DataFrame.

- Return: The number of rows removed.

4. Find Highest Temp [4 Marks]

def find_highest_temp(self) -> int:

- Logic: Find the maximum temperature value across all cleaned records.
- Return: The peak integer temperature.

5. Average Temperature per Sensor [4 Marks]

def sensor_averages(self) -> dict:

- Logic: Calculate mean 'Temperature' for each 'SensorID' (rounded to 2 decimal places).
- Return: Dictionary with 'SensorID' as keys and average temperatures as values.

6. High Temperature Sensors [4 Marks]

def high_temp_sensors(self, threshold: int) -> list:

- Logic: Identify unique 'SensorID' values who have at least one reading above 'threshold'.
- Return: Sorted list of unique high-temperature sensor IDs.

7. Count High Temperature Sensors [4 Marks]

def count_high_temp_sensors(self, threshold: int) -> int:

- Logic: Count how many unique sensors meet the high-temperature criteria (> 'threshold').
- Return: Integer count of high-temperature sensors.

Test Cases & Marks Allocation

Test Case ID	Description	Method	Marks
TC1	Verify initial state (Sample)	__init__	0
TC2	Verify data loading (Sample)	read_data()	0
TC3	Verify removal of missing values	clean_records()	4
TC4	Identify highest recorded temp	find_highest_temp()	4
TC5	Calculate averages per sensor	sensor_averages()	4
TC6	Identify sensors exceeding 40 degrees	high_temp_sensors(40)	4
TC7	Count unique high-temperature sensors	count_high_temp_sensors(40)	4

Visible Test Case Descriptions

- TC1: self.df is None upon class creation.
- TC2: read_data() loads the sample data into self.df (12 rows initially).
- TC3: clean_records() returns 2 (Removing records for S02 and S06).
- TC4: find_highest_temp() returns 45.
- TC5: sensor_averages()['S01'] returns approximately 32.33.
- TC6: high_temp_sensors(40) returns ['S03'].
- TC7: count_high_temp_sensors(40) returns 1.